

**REPÚBLICA BOLIVARIANA DE VENEZUELA**  
**MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA**  
**UNIVERSIDAD NACIONAL EXPERIMENTAL DE LAS TELECOMUNICACIONES E**  
**INFORMÁTICA**



**EVALUACIÓN 3 — BASES DE DATOS**

Proyecto: Cine UNETI — Sistema CRUD con Angular, Node.js y MongoDB

Asignatura: Bases de Datos

**Autor:**  
Miguel Figuera  
C.I.: 23.558.789

La Victoria, julio, 2026

## ÍNDICE

1. Enlace del Repositorio de GitHub
2. Descripción del Proyecto
3. Instrucciones de Ejecución (README.md)
4. Comandos para Visualizar el Clúster y las Colecciones
5. Capturas de MongoDB — Conexión y Colecciones
6. Colecciones Verificadas

## 1. Enlace del Repositorio de GitHub

El código fuente completo del proyecto se encuentra publicado en GitHub con visibilidad pública, de modo que puede consultarse y clonarse sin autenticación. El repositorio contiene el backend Node.js con Express, el frontend Angular, los scripts de poblado de la base de datos y el archivo README.md con la documentación de ejecución.

### *Repositorio público*

```
https://github.com/miguelfiguera/angular-crud-uneti  
Visibilidad: público · Rama principal: main
```

## 2. Descripción del Proyecto

Cine UNETI es un sistema CRUD full-stack para la gestión de un catálogo de películas. La interfaz fue construida con Angular 19 y Tailwind CSS; la API REST con Node.js y Express; y la persistencia con MongoDB, en la base de datos cine\_uneti, que contiene cinco colecciones con al menos cuatro campos cada una: películas (título, año, género, duración y calificación), directores, generos, actores y resenas. Desde la interfaz es posible añadir películas a la lista, consultarlas, eliminarlas y ejecutar una consulta sencilla por género que se resuelve directamente en MongoDB.

## 3. Instrucciones de Ejecución (README.md)

El README.md del repositorio documenta la descripción del proyecto y el procedimiento completo de puesta en marcha: clonación del repositorio, configuración de las variables de entorno, instalación de dependencias (que ejecuta automáticamente el seed de la base de datos) y arranque simultáneo de la API y del frontend. El proyecto usa Yarn como gestor de paquetes; los comandos npm install y npm start equivalen a yarn install y yarn start.

### ***Puesta en marcha del proyecto***

```
# 1. Clonar el repositorio
git clone https://github.com/miguelfiguera/angular-crud-uneti.git
cd angular-crud-uneti

# 2. Configurar variables de entorno
cp .env.example .env
#   MONGODB_URI=mongodb://localhost:27017
#   MONGODB_DB=cine_uneti
#   PORT=3000

# 3. Instalar dependencias (ejecuta el seed automáticamente)
yarn install          # equivalente: npm install

# 4. Iniciar API (localhost:3000) y frontend (localhost:4200)
yarn start            # equivalente: npm start

# Poblar la base de datos manualmente si hiciera falta
yarn seed
```

## **4. Comandos para Visualizar el Clúster y las Colecciones**

Para tomar las capturas exigidas por la evaluación —donde se vean la conexión y las cinco colecciones— pueden usarse dos vías. La primera es MongoDB Compass (o la vista Collections del clúster en Atlas): basta con conectarse con la cadena de conexión y abrir la base de datos cine\_uneti, donde las cinco colecciones aparecen en el panel izquierdo con sus documentos.

### ***Conexión en MongoDB Compass / Atlas***

```
# Cadena de conexión local (Compass → New Connection)
mongodb://localhost:27017

# Con MongoDB Atlas, usar la URI del clúster:
mongodb+srv://<usuario>:<password>@<cluster>.mongodb.net

# Luego: base de datos cine_uneti → se listan las 5 colecciones
# (películas, directores, géneros, actores, reseñas)
```

La segunda vía es la terminal con mongosh, que fue la utilizada en este trabajo. Los siguientes comandos muestran la conexión activa, el nombre de la base de datos, la lista de colecciones, el conteo de documentos por colección y las estadísticas generales.

### Comandos de verificación en mongosh

```
mongosh "mongodb://localhost:27017/cine_uneti"

db.getMongo()           // conexión activa al clúster
db.getName()            // → cine_uneti
db.getCollectionNames() // → las 5 colecciones
db.getCollectionInfos() // detalle de cada colección
db.peliculas.countDocuments()
db.peliculas.find()      // documentos de la colección
db.stats()               // resumen: 5 colecciones, objetos, tamaños

// Conteo de todas las colecciones en una sola línea:
mongosh cine_uneti --quiet --eval "
  print('Base de datos: ' + db.getName());
  db.getCollectionNames().forEach(c =>
    print(c + ': ' + db[c].countDocuments() + ' documentos'));"
```

Todo lo anterior quedó automatizado en el script `scripts/mongo-test.sh` del repositorio, registrado como `yarn mongo-test`, que ejecuta en secuencia los comandos nativos de `mongosh` sobre las cinco colecciones e imprime cada comando junto a su salida. Las capturas de la sección siguiente corresponden a esa ejecución.

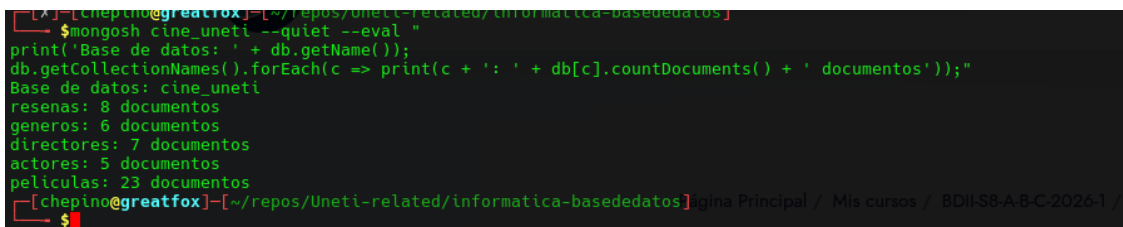
### Script del repositorio

```
yarn mongo-test      # ejecuta bash scripts/mongo-test.sh

# El script recorre las 5 colecciones y ejecuta:
#   db.getMongo(), db.getName(), db.getCollectionNames(),
#   db.getCollectionInfos(), countDocuments(), findOne(),
#   find() y db.stats()
```

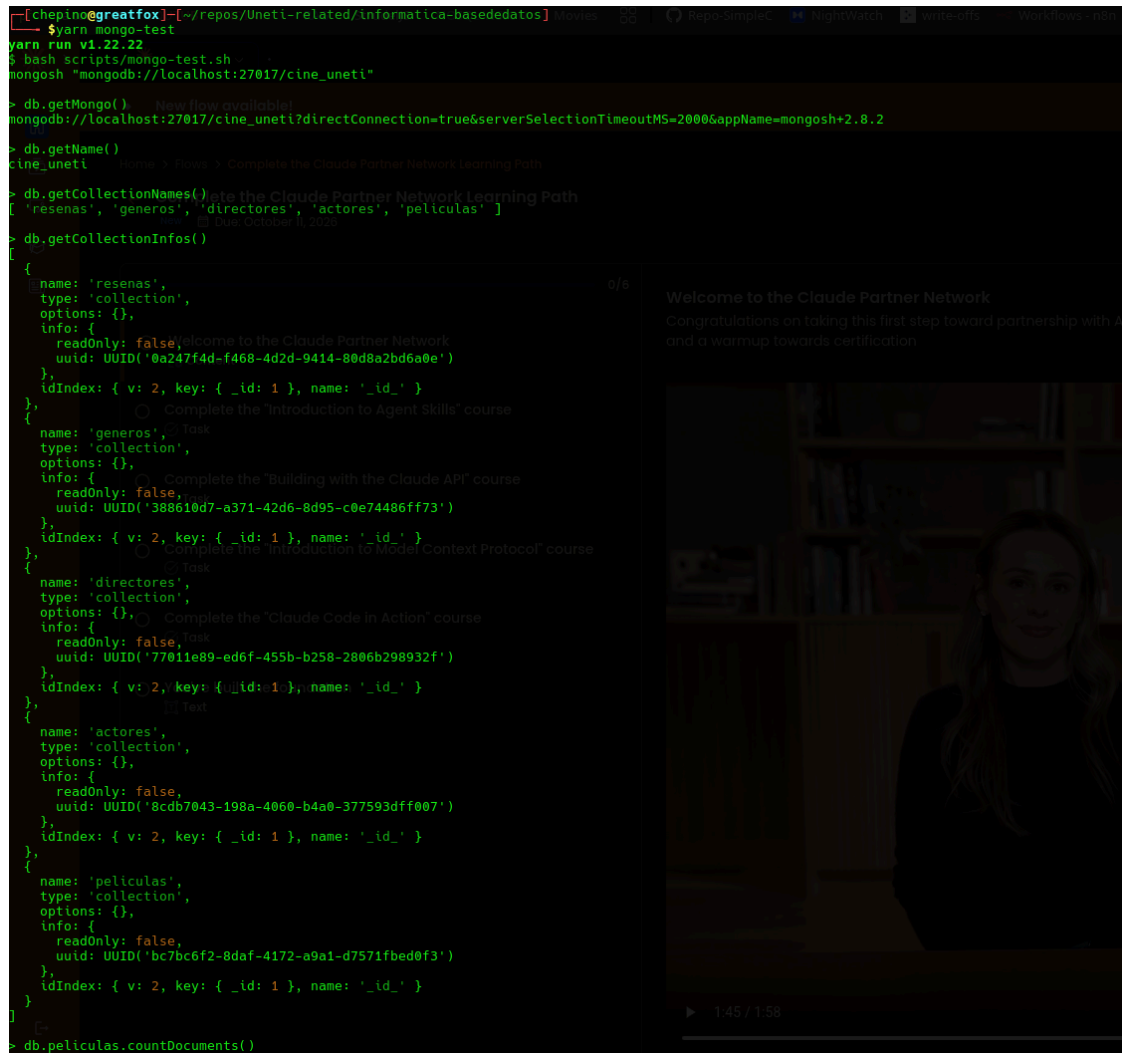
## 5. Capturas de MongoDB — Conexión y Colecciones

Las siguientes capturas de pantalla evidencian la conexión al servidor de MongoDB y la existencia de las cinco colecciones con sus documentos, con la salida nativa de `mongosh` generada por `yarn mongo-test`.



```
[*] [cheptino@greatfox] [~/repos/Uneti-related/informatica-basededatos]
$ mongosh cine_uneti --quiet --eval "
print('Base de datos: ' + db.getName());
db.getCollectionNames().forEach(c => print(c + ': ' + db[c].countDocuments() + ' documentos'));"
Base de datos: cine_uneti
resenas: 8 documentos
generos: 6 documentos
directores: 7 documentos
actores: 5 documentos
peliculas: 23 documentos
[cheptino@greatfox] [~/repos/Uneti-related/informatica-basededatos] igina Principal / Mis cursos / BDII-S8-A-B-C-2026-1 /
```

Figura 1. Conteo de documentos de las cinco colecciones de cine\_uneti con mongosh --eval en una sola línea.



```

[chepino@greatfox] ~/repos/Uneti-related/informatica-basededatos
$ yarn mongo-test
yarn run v1.22.22
$ bash scripts/mongo-test.sh
mongosh "mongodb://localhost:27017/cine_uneti"

> db.getMongo()
New flow available!
mongodb://localhost:27017/cine_uneti?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2

> db.getName()
cine_uneti

> db.getCollectionNames()
[ 'resenas', 'generos', 'directores', 'actores', 'peliculas' ]

> db.getCollectionInfos()
[
  {
    name: 'resenas',
    type: 'collection',
    options: {},
    info: {
      readOnly: false,
      uuid: UUID('0a247f4d-f468-4d2d-9414-80d8a2bd6a0e')
    },
    idIndex: { v: 2, key: { _id: 1 }, name: '_id_' }
  },
  {
    name: 'generos',
    type: 'collection',
    options: {},
    info: {
      readOnly: false,
      uuid: UUID('388610d7-a371-42d6-8d95-c0e74486ff73')
    },
    idIndex: { v: 2, key: { _id: 1 }, name: '_id_' }
  },
  {
    name: 'directores',
    type: 'collection',
    options: {},
    info: {
      readOnly: false,
      uuid: UUID('77011e89-ed6f-455b-b258-2806b298932f')
    },
    idIndex: { v: 2, key: { _id: 1 }, name: '_id_' }
  },
  {
    name: 'actores',
    type: 'collection',
    options: {},
    info: {
      readOnly: false,
      uuid: UUID('8cdb7043-198a-4060-b4a0-377593dff007')
    },
    idIndex: { v: 2, key: { _id: 1 }, name: '_id_' }
  },
  {
    name: 'peliculas',
    type: 'collection',
    options: {},
    info: {
      readOnly: false,
      uuid: UUID('bc7bc6f2-8daf-4172-a9a1-d7571fbed0f3')
    },
    idIndex: { v: 2, key: { _id: 1 }, name: '_id_' }
  }
]

> db.peliculas.countDocuments()

```

Figura 2. Ejecución de yarn mongo-test: conexión con db.getMongo(), base de datos activa con db.getName() y colecciones con db.getCollectionNames() y db.getCollectionInfos().

```

> db.peliculas.countDocuments()
23

> db.peliculas.findOne()
{
  _id: ObjectId('6a54335c7be321e436143085'),
  titulo: 'El Padrino',
  anio: 1972,
  genero: 'Drama',
  duracionMinutos: 175,
  calificacion: 9.2,
  imageUrl: 'https://image.tmdb.org/t/p/w500/3bhkrj58Vtu7enYsRolD1fZdja1.jpg'
}

> db.peliculas.find()
{
  _id: ObjectId('6a54335c7be321e436143085'),
  titulo: 'El Padrino',
  anio: 1972,
  genero: 'Drama',
  duracionMinutos: 175,
  calificacion: 9.2,
  imageUrl: 'https://image.tmdb.org/t/p/w500/3bhkrj58Vtu7enYsRolD1fZdja1.jpg'
},
{
  _id: ObjectId('6a54335c7be321e436143086'),
  titulo: 'Pulp Fiction',
  anio: 1994,
  genero: 'Crimen',
  duracionMinutos: 154,
  calificacion: 8.9,
  imageUrl: 'https://image.tmdb.org/t/p/w500/vQWk5YBFWF4bZaofAbvEtShwBvQ.jpg'
},
{
  _id: ObjectId('6a54335c7be321e436143087'),
  titulo: 'El Señor de los Anillos',
  anio: 2001,
  genero: 'Fantasía',
  duracionMinutos: 178,
  calificacion: 8.8,
  imageUrl: 'https://image.tmdb.org/t/p/w500/6oom5QYQ2yQTMJIBnvbkBL9chO6.jpg'
},
{
  _id: ObjectId('6a54335c7be321e436143088'),
  titulo: 'Matrix',
  anio: 1999,
  genero: 'Ciencia Ficción',
  duracionMinutos: 136,
  calificacion: 8.7,
  imageUrl: 'https://image.tmdb.org/t/p/w500/dXNAPwY7VrqMao51EKhhCJfaGb5.jpg'
},
{
  _id: ObjectId('6a54335c7be321e436143089'),
  titulo: 'Parásitos',
  anio: 2019,
  genero: 'Drama',
  duracionMinutos: 132,
  calificacion: 8.6,
  imageUrl: 'https://image.tmdb.org/t/p/w500/7I1TTglo3zvGI1TAIymCfbf13vt.jpg'
},
{
  _id: ObjectId('6a54335c7be321e43614308a'),
  titulo: 'El mundo encerrado',
  anio: 1998,
  genero: 'Anime',
  duracionMinutos: 1,
  calificacion: 6.5,
  imageUrl: 'https://image.tmdb.org/t/p/w500/3Z93fUMdcEQMyhtyTXwu8FUPaGj.jpg'
}

```

Figura 3. Información detallada de las colecciones con `db.getCollectionInfos()`.

```

]
Type "it" for more
> db.directores.countDocuments()
7
> db.directores.findOne()
{
  _id: ObjectId('6a54335c7be321e43614309c'),
  nombre: 'Francis Ford Coppola',
  nacionalidad: 'Estados Unidos',
  peliculasDirigidas: 37,
  premiosOscar: 5
}
> db.directores.find()
[
  {
    _id: ObjectId('6a54335c7be321e43614309c'),
    nombre: 'Francis Ford Coppola',
    nacionalidad: 'Estados Unidos',
    peliculasDirigidas: 37,
    premiosOscar: 5
  },
  {
    _id: ObjectId('6a54335c7be321e43614309d'),
    nombre: 'Quentin Tarantino',
    nacionalidad: 'Estados Unidos',
    peliculasDirigidas: 10,
    premiosOscar: 2
  },
  {
    _id: ObjectId('6a54335c7be321e43614309e'),
    nombre: 'Makoto Shinkai',
    nacionalidad: 'Japón',
    peliculasDirigidas: 13,
    premiosOscar: 0
  },
  {
    _id: ObjectId('6a54335c7be321e43614309f'),
    nombre: 'Elia Schneider',
    nacionalidad: 'Venezuela',
    peliculasDirigidas: 8,
    premiosOscar: 0
  },
  {
    _id: ObjectId('6a54335c7be321e4361430a0'),
    nombre: 'Miguel Ferrari',
    nacionalidad: 'Venezuela',
    peliculasDirigidas: 4,
    premiosOscar: 0
  },
  {
    _id: ObjectId('6a54335c7be321e4361430a1'),
    nombre: 'Efterpi Charalambidis',
    nacionalidad: 'Venezuela',
    peliculasDirigidas: 3,
    premiosOscar: 0
  }
]

```

Figura 4. Conteo y documentos de las colecciones (`countDocuments`, `findOne` y `find`).

```

> db.stats()
{
  db: 'cine_uneti',
  collections: Long('5'),
  views: Long('0'),
  objects: Long('49'),
  avgObjSize: 165.81632653061226,
  dataSize: 8125,
  storageSize: 184320,
  indexes: Long('5'),
  indexSize: 184320,
  totalSize: 368640,
  scaleFactor: Long('1'),
  fsUsedSize: 3088703488,
  fsTotalSize: 12179079168,
  ok: 1
}
Done in 4.56s.

```

Figura 5. Resumen `db.stats()` de `cine_uneti`: 5 colecciones y 49 objetos, cierre de `yarn mongo-test`.

## 6. Colecciones Verificadas

La verificación con mongosh confirmó la base de datos cine\_uneti con sus cinco colecciones pobladas y los siguientes conteos de documentos al momento de la captura:

- películas — 23 documentos
- directores — 7 documentos
- géneros — 6 documentos
- actores — 5 documentos
- reseñas — 8 documentos